

Introduction to Arrays (1D & 2D)

Dr. S. M. Farooq

Assistant Professor Senior

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

Introduction to Pointers

Definition: A pointer is a variable which holds only the address of another variable.

Syntax: <datatype> *variable_name(s);

E.g. `int *a, b=20;`

`a=&b;`

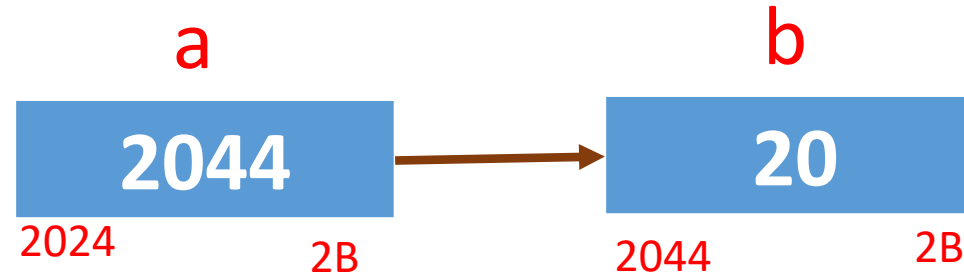
`printf("\n%d",b);`

`printf("\n%d",&b);`

`printf("\n%d",a);`

`printf("\n%d",&a);`

`printf("\n%d",*a);`



E.g. `int *a;`

`float b=20;`

`a=&b;` ✗

Both variables data types should be same.

`*a` => valueat(a), `*` is dereferencing operator

=> valueat(2044)

=> 20

Accessing the value of b using pointer variable a.

Introduction to Arrays

- An array is a collection of similar elements. That is data types are same.

Syntax: Datatype variable_name[Size];

Declaration:

E.g. `int a[5];`

E.g. `float b[5];`

- Datatype can be any type of data supported in C.
- Variable_name is any valid Identifier name.
- Size is maximum number of elements that can be stored in the array.
- Here, memory is allocated consecutively.
- Array Indexing always starts with zero.

Initialization

E.g. `int a[5]={10, 20, 30, 40, 50};`

Accessing Array elements

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50
2020	2022	2024	2026	2028

```
int i, a[5]={10, 20, 30, 40, 50};  
for( i=0; i<=4; i++)  
    printf(" %d", a[i]);
```

C program to take a set of elements input and display using arrays

```
#include<stdio.h>
int main( )
{
    int a[5],i;
    printf("\n Enter array elements:");
    for(i=0; i<=4; i++)
        scanf("%d",&a[i]);
    printf("\n Array elements are:");
    for( i=0; i<=4; i++)
        printf(" %d", a[i]);
    return 0;
}
```

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50

C program to print sum of the Given finite number of Integers

```
#include<stdio.h>
int main( )
{
    int a[5], i, sum=0;
    printf("\n Enter array elements:");
    for(i=0; i<=4; i++)
        scanf("%d",&a[i]);
    for( i=0; i<=4; i++)
        sum=sum+ a[i];
    printf(" sum=%d", sum);
    return 0;
}
```

Tracing

Iteration	i=0	i<=4	Sum=sum+a[i]	i = i + 1
Iter1	0	0<=4 T	Sum = 0 + 10=10	i = 0+1
Iter2	1	1<=4 T	Sum = 10 + 20=30	i = 1+1
Iter3	2	2<=4 T	Sum = 30 + 30=60	i = 2+1
Iter4	3	3<=4 T	Sum = 60 + 40=100	i = 3+1
Iter5	4	4<=4 T	Sum = 100 + 50=150	i = 4+1
Iter6	5	5<=4 F	-	-

Input

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50

C program to find smallest number from the given finite number of Integers

```
#include<stdio.h>
int main( )
{
    int a[5], i, small;
    printf("\n Enter array elements:");
    for(i=0; i<=4; i++)
        scanf("%d",&a[i]);
    small=a[0];
    for( i=1; i<=4; i++)
    {
        if (a[i] < small)
            small=a[i];
    }
    printf(" small=%d", small);
    return 0;
}
```

Tracing

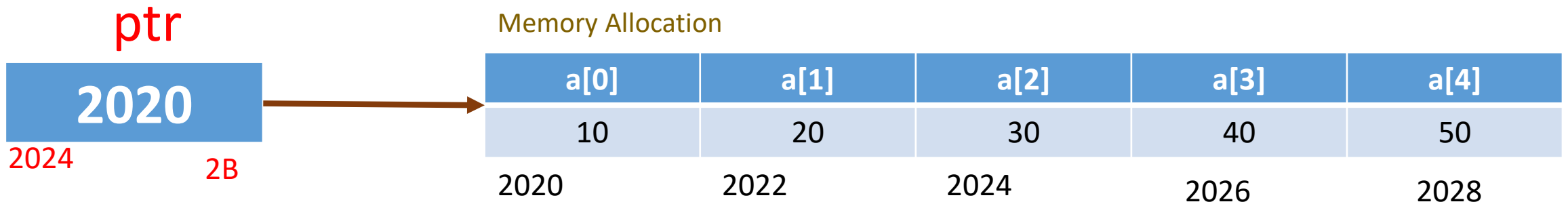
Iteration	Small =a[0], i=1	i<=4	If (a[i] < small) ?	Small=a[i]	i = i + 1
Iter1	10,i=1	1<=4 T	20 < 10 F	-	i = 1+1
Iter2	i=2	2<=4 T	15 < 10 F	-	i = 2+1
Iter3	i=3	3<=4 T	12 < 10 F	-	i = 3+1
Iter4	i=4	4<=4 T	05 < 10 T	Small=05	i = 4+1
Iter5	i=5	5<=4 F	-	-	-

Input

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	15	12	05

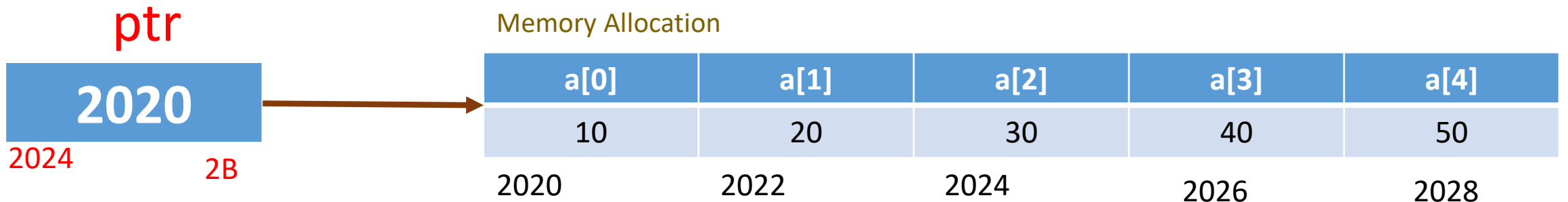
Pointers with Arrays

```
int a[5]={ 10, 20, 30, 40, 50}, *ptr;  
ptr=&a[0]; // it is the starting address of array a.  
printf("%d",*ptr); // *ptr => valueat(ptr) => valueat(2020) => 10  
ptr = ptr +1 ; // when we increment an address, it will be incremented by its size.  
                ptr + 1 * scaling_factor => 2020 + 1*2 = 2022  
printf("%d",*ptr); // *ptr => valueat(ptr) => valueat(2022) => 20  
ptr = ptr +1 ; // 2024 + 1*2 = 2024  
printf("%d",*ptr); // *ptr => valueat(ptr) => valueat(2024) => 30
```



Pointers with Arrays cntd..

```
int a[5]={ 10, 20, 30, 40, 50}, *ptr, i=1;
ptr=a; // The array name represents starting address of array a => &a[0].
while(i<=5)
{
    printf("%d",*ptr);
    ptr = ptr +1 ;
    i++;
}
```



The real thing

```
int num[ ] = { 24, 34, 12, 44, 56, 17 };
```

The name of the an array refer to its base address. Thus, by saying * num, we would be able to refer to the zeroth element of the array, that is 24. Further, the following also true.

$*(num+0) \rightarrow num[0]$

$*(num+1) \rightarrow num[1]$

...

$*(num+i) \rightarrow num[i]$

The following expressions are also true. $num[i]$ $*(num + i)$ $*(i + num)$ $i[num]$

C- program to find a number and its frequency

```
#include<stdio.h>
Int main( )
{
    int num[ ] = { 7, 3, 5, 4, 6, 7, 2, 4, 6, 7 };
    int n, i, count;
    printf("\n Enter an element to search:");
    scanf("%d",&n);
    count =0 ;
    for ( i=0; i <= 9 ; i++)
    {
        if( num[i] == n)
            count++;
    }
    printf("\n Number %d is found %d time(s) in the array\n", n, count);
    return 0;
}
```

Two-Dimensional Arrays

The two dimensional (2-D) array is called a matrix.

Declaration:

```
int array_name[ row_dimension][column_dimension];
```

Array_name is the name of the 2-D array, where as row_dimension and column dimension refers to number of rows and columns in the matrix respectively.

Initialization:

```
int stud [4] [2] = {  
                    {1234, 56, }, { 1212, 33}, { 1434, 80}, { 1312, 78}  
                    };
```

[or]

```
int stud[4][2] = {1234, 56, 1212, 33, 1434, 80, 1312, 78};
```

Two-Dimensional Arrays cntd..

While initializing a 2-D array, it is necessary to mention the second (column) dimension, whereas the first dimension (row) is optional. Thus, the following declarations are valid.

```
int a[2][3] = { 10, 11, 19, 14, 59, 90 };
```

```
Int a[ ][3]={10, 11, 19, 14, 59, 90 };
```

	0	1	2
0	10	11	19
1	14	59	90

Memory Map of a 2-D Array:

```
int s[4][2]={101, 12, 102, 14, 103, 16, 104, 18};
```

s[0][0]	s[0][1]	s[1][0]	s[1][1]	s[2][0]	s[2][1]	s[3][0]	s[3][1]
101	12	102	14	103	16	104	18
20000	20004	20008	20012	20016	20020	20024	20028